Printing from a FireMonkey Application

Go Up to FireMonkey Applications Guide

You can print from a FireMonkey application on either **Windows** or **OS X**.

Contents

FireMonkey Printing API

The FMX.Printer.TPrinter object encapsulates the functionality for accessing and printing to a printer attached to a computer on either Windows or OS X. Use the FMX.Printer.TPrinter object to manage any printing performed by an application. Obtain an instance of FMX.Printer.TPrinter by calling the global FMX.Printer.Printer function.

FMX.Printer.TPrinter has a list of TPrinterDevice objects, each representing a printer. Each printer has its own list of printing resolutions in DPI (dots per inch). The DPICount property gives you the number of resolutions that the printer supports. The DPI property is an indexed property that can be traversed to find out the actual resolutions. ActiveDPIIndex is a zero-based index for the DPI indexed property that represents the printing resolution on the printing device.

Enabling Printing in Your FireMonkey Application

The following steps show you how to print successfully from a FireMonkey application.

1. Include FMX.Printer in the form that issues the printing job:

   Delphi:

   ```
   uses FMX.Printer;
   ```

   C++:

   ```
   #include "FMX.Printer.hpp"
   ```

2. Always use the printer canvas or a graphical component's canvas (for instance, the canvas of an image) instead of the form's canvas. When not using the printer canvas, always start the printing operation with BeginScene and end the printing job with EndScene.
3. For full control over the printing quality, the Canvas resolution and portability, set ActiveDPIIndex, in the first place, either by assigning it a value or by calling the SelectDPI method (the recommended way):

   Delphi:

```
Printer.ActivePrinter.SelectDPI(1200, 1200); { one way }
Printer.ActivePrinter.ActiveDPIIndex := 1; { another way }
```

C++:

```
Printer->ActivePrinter->SelectDPI(1200, 1200); /* one way */
Printer->ActivePrinter->ActiveDPIIndex = 1; /* another way */
```

4. Start the actual printing job with BeginDoc:
   Delphi:

```
Printer.BeginDoc;
```

C++:

```
Printer->BeginDoc();
```

5. End the actual printing job with EndDoc:
   Delphi:

```
Printer.EndDoc;
```

C++:

```
Printer->EndDoc();
```

6. Never change ActiveDPIIndex during a printing job.

About DPI and Driver Support

The driver support for the same printer varies greatly on different platforms (Windows, OS X). As a result, you should not depend on the fact that the number or order of supported resolutions in the DPI array is the same between Windows and OS X. For instance, there are only 3 supported resolutions for the HP Laser Printer on OS X, whereas there are 7 resolutions for Windows. If you have applications with printing functionality on both Windows and OS X, you need to add additional code (mainly specifying the DPI) to ensure that the printing output is reasonably similar on both platforms.

The **best practice** is always to specify the DPI either by setting the ActiveDPIIndex property or calling the SelectDPI method, before printing, as mentioned in the steps above.

Note that ActiveDPIIndex is **not set** to the default DPI for a given printer. A TPrinterDevice object does not support a default DPI, especially because some OS X printer drivers do not report the default DPI. Because of this, ActiveDPIIndex is set to -1 when the application starts, for all TPrinterDevice objects. The value -1 means that it will use either the last printing DPI or the default DPI. The default DPI is available only if after the application started, you did not change ActiveDPIIndex to some other index value and you did not call BeginDoc.

On the other hand, you can set ActiveDPIIndex to -1 and the application will use the last DPI of the printer that was previously set. So always remember to set a value to ActiveDPIIndex before calling BeginDoc, because you want the same Canvas size on both Windows and OS X.

The SelectDPI method is very easy to use to let a TPrinter object select the closest resolution to that of your printer, based on the parameters passed to SelectDPI. It is very easy to set; for instance, instead of directly setting the ActiveDPIIndex property, you can change the value at run time:

Delphi:

```
Printer.ActivePrinter.Select(600, 600);
```

C++:

```
Printer->ActivePrinter->Select(600, 600);
```

Given the fact that, on Windows, the number of printer resolutions reported by the driver differs from the one reported by the OS X printer driver, the specified index might not be available.

In conclusion, using SelectDPI is a **more convenient way** of setting the printing DPI.

Printer Canvas

There are a few differences in the usage of a **printer canvas** compared to a **screen or a window (form) canvas**:

1. When using the printer canvas, you must set the color and fill preferences using Canvas.Fill:
   Delphi:

   ```
   Canvas.Fill.Color := claXXX;
   Canvas.Fill.Kind := TBrushKind.Solid;
   ```

   C++:

   ```
   Canvas->Fill->Color = claXXX;
   Canvas->Fill->Kind = TBrushKind.Solid;
   ```

2. When using the screen or window canvas, you must set the color and fill preferences using Canvas.Stroke:
   Delphi:

   ```
   Canvas.Stroke.Color := claXXX;
   Canvas.Stroke.Kind := TBrushKind.Solid;
   ```

   C++:

   ```
   Canvas->Stroke->Color = claXXX;
   Canvas->Stroke->Kind = TBrushKind.Solid;
   ```

Keep in mind that:

- When you are printing text and you want to change its color, use Canvas.Fill.
- Wen you are drawing anything else, excepting text, use Canvas.Stroke.

The `Opacity` parameter, for all the routines that support it (such as FMX.Graphics.TCanvas.FillRect), takes floating-point values in the range `0..1` (where 0 is transparent and 1 is opaque). Do not set values outside this range.

Example of Programmatic Printing

The following example uses an image and a button. Whenever you click the button, the image is printed to the printer.

Delphi:

```
procedure TMyForm.PrintButtonClick(Sender: TObject);
var
  SrcRect, DestRect: TRectF;

begin
  { Set the default DPI for the printer. The SelectDPI routine defaults
    to the closest available resolution as reported by the driver. }
  Printer.ActivePrinter.SelectDPI(1200, 1200);

  { Set canvas filling style. }
  Printer.Canvas.Fill.Color := claBlack;
  Printer.Canvas.Fill.Kind := TBrushKind.Solid;

  { Start printing. }
  Printer.BeginDoc;

  { Set the Source and Destination TRects. }
  SrcRect := Image1.LocalRect;
  DestRect := TRectF.Create(0, 0, Printer.PageWidth, Printer.PageHeight);

  { Print the picture on all the surface of the page and all opaque. }
  Printer.Canvas.DrawBitmap(Image1.Bitmap, SrcRect, DestRect, 1);

  { Finish printing job. }
  Printer.EndDoc;
end;
```

C++:

```
void __fastcall TMyForm::PrintButtonClick(TObject *Sender)
{
  TRectF SrcRect, DestRect;
  TPrinter *Printer = Printer;

  /* Set the default DPI for the printer. The SelectDPI routine defaults
         to the closest available resolution as reported by the driver. */
  Printer->ActivePrinter->SelectDPI(1200, 1200);
```

```
  /* Set canvas filling style. */
  Printer->Canvas->Fill->Color = claBlack;
  Printer->Canvas->Fill->Kind = TBrushKind(1);

  /* Start printing. */
  Printer->BeginDoc();

  /* Set the Source and Destination TRects. */
  SrcRect = Image1->LocalRect;
  DestRect = TRectF(0, 0, Printer->PageWidth, Printer->PageHeight);

  /* Print the picture on all the surface of the page and all opaque. */
  Printer->Canvas->DrawBitmap(Image1->Bitmap, SrcRect, DestRect, 1);

  /* Finish the printing job. */
  Printer->EndDoc();
}
```

For more information regarding the printing API, please refer to the API documentation in the FMX.Printer unit.

See Also

- FMX.Printer Unit

Code Examples

- FireMonkey Printing (Delphi)
- FireMonkey Printing (C++)
